

Prompt B — Habitaciones UI: Arrivals vs In-House, Planning abre Form Completo, Filtro Histórico

PARTE 1 — Distinguir “Ingresa hoy” vs “Ya en casa” en el Planning

El objetivo

Una habitación ocupada que hizo check-in ayer se ve igual que una que hace check-in hoy. Hay que diferenciarlas visualmente.

1a — Agregar status `checkin_today` en `shared/schema.ts`

```
// Reemplazar:
export type PlanningCellStatus = "available" | "booked" | "checked_in" | "checkout"

// Por:
export type PlanningCellStatus = "available" | "booked" | "checked_in" | "checkin"
```

1b — Asignar `checkin_today` en el backend de planning (DB)

En `server/storage.ts`, en el método `getPlanningData` (la versión DB, no la in-memory), dentro del loop que asigna estados de occupancy:

```
// Buscar el bloque que dice:
} else if (reservation.status === "checked_in") {
  occupancy[room.id].push("checked_in");
} else {
  occupancy[room.id].push("booked");
}

// Reemplazar por:
} else if (reservation.status === "checked_in") {
  if (day === reservation.checkOutDate) {
    occupancy[room.id].push("checkout_today");
  } else if (day === reservation.checkInDate) {
    occupancy[room.id].push("checkin_today");
  }
}
```

```

    } else {
      occupancy[room.id].push("checked_in");
    }
  } else {
    occupancy[room.id].push("booked");
  }
}

```

Hacer lo mismo en la versión **in-memory** del mismo método (hay dos implementaciones en el archivo).

1c — Color y label para `checkin_today` en `planning.tsx`

En `getStatusColor`, agregar case:

```

case "checkin_today":
  return "bg-green-200 dark:bg-green-800/60 border-green-400 dark:border-green-60

```

En `getStatusLabel`, agregar case:

```

case "checkin_today":
  return "Ingresa hoy";

```

1d — Agregar a la leyenda `statusItems`

```

// Agregar en el array statusItems, después de "booked":
{ status: "checkin_today", label: "Ingresa hoy" },

```

PARTE 2 — Click en celda vacía del Planning abre el formulario completo de reserva

Situación actual

Al hacer click en una celda libre del planning se abre `QuickReservationDialog`, un formulario simplificado. El equipo prefiere que abra el mismo formulario completo que usa el botón “Nueva Reserva”.

Fix en `planning.tsx`

2a — Agregar estado para el form completo

```
// Ya existe en planning.tsx:
const [editReservationOpen, setEditReservationOpen] = useState(false);
const [editingReservationData, setEditingReservationData] = useState<ReservationW

// Agregar:
const [newReservationOpen, setNewReservationOpen] = useState(false);
const [newReservationDefaults, setNewReservationDefaults] = useState<{ roomId?: s
```

2b — Cambiar `handleCellClick` para abrir el form completo

```
const handleCellClick = (room: RoomWithType, day: string, status: PlanningCellSta
  if (status === "available") {
    // En lugar de abrir QuickReservationDialog, abrir el form completo con los d
    setNewReservationDefaults({
      roomId: room.id,
      roomIdType: room.roomTypeId,
      checkInDate: day,
    });
    setNewReservationOpen(true);
  } else {
    const resolvedId = reservationId || findReservationForRoomAndDay(room.id, day
    if (resolvedId) {
      setSelectedReservationId(resolvedId);
      setReservationDetailOpen(true);
    }
  }
};
```

2c — Montar `ReservationFormDialog` para nueva reserva

Agregar en el JSX del componente (junto al `ReservationFormDialog` de edición que ya existe):

```
<ReservationFormDialog
  reservation={undefined}
  guests={guests}
  rooms={allRooms}
  roomTypes={roomTypes}
  open={newReservationOpen}
  onOpenChange={ (open) => {
    setNewReservationOpen(open);
    if (!open) setNewReservationDefaults(null);
  }}
  defaultValues={newReservationDefaults || undefined}
```

```

onSuccess={() => {
  setNewReservationDefaults(null);
  queryClient.invalidateQueries({ queryKey: ["/api/planning"] });
  queryClient.invalidateQueries({ queryKey: ["/api/reservations"] });
}}
/>

```

2d — Agregar prop `defaultValues` a `ReservationFormDialog`

En el componente `ReservationFormDialog` (en `reservations.tsx` o donde esté definido), agregar la prop opcional:

```

interface ReservationFormDialogProps {
  // ... props existentes ...
  defaultValues?: {
    roomId?: string;
    roomIdType?: string;
    checkInDate?: string;
  };
}

```

Y en el `useEffect` que inicializa el form cuando se abre:

```

useEffect(() => {
  if (open && !reservation) {
    // Nuevo form - aplicar defaults del planning si los hay
    setFormData(prev => ({
      ...prev,
      roomId: defaultValues?.roomId || prev.roomId || "",
      roomIdType: defaultValues?.roomIdType || prev.roomIdType || "",
      checkInDate: defaultValues?.checkInDate || prev.checkInDate || today,
    }));
  }
}, [open, reservation, defaultValues]);

```

2e — Eliminar `QuickReservationDialog` del JSX (opcional)

Como ya no se usa, se puede comentar o eliminar el montaje de

`QuickReservationDialog`. Los estados `quickReservationOpen` y `selectedCell` también se pueden eliminar para limpiar el código.

PARTE 3 — Filtro histórico en Check-in, Check-out y Grupos

El problema

Las páginas muestran todas las reservas sin fecha límite. Necesitan mostrar solo las activas/del día por defecto, con opción de ver el historial.

3a — CheckInPage: filtro y selector de fecha para histórico

La página ya tiene una tab “Historial” con `historyDate`. Verificar que funcione correctamente y que la tab “Reservas” (la principal) solo muestre check-ins de hoy y mañana (ya lo hace via `/api/reservations/check-in`).

Agregar formato de fecha correcto en la lista. Buscar donde se muestra `reservation.checkInDate` en la lista de check-in y aplicar `formatDateAR`:

```
// Buscar renderizado de fechas en check-in list y reemplazar raw dates:
{formatDateAR(reservation.checkInDate)}
```

(Importar o definir `formatDateAR` como: `(d: string) => d ? d.split("-").reverse().join("/") : "-"`)

3b — CheckOutPage: filtro por defecto — solo hoy + opción ver todos

La query usa `/api/reservations/check-out` que devuelve todos los `checked_in`. Esto puede incluir reservas de días anteriores que nunca se procesaron.

Agregar filtro frontend:

```
const today = new Date().toLocaleDateString("en-CA", { timeZone: "America/Argenti

const filteredCheckouts = reservations?.filter(res => {
  const matchesSearch = !searchQuery ||
    `${res.guest?.firstName} ${res.guest?.lastName}`.toLowerCase().includes(search
    res.room?.roomNumber?.includes(searchQuery);

  if (showAllCheckouts) return matchesSearch;

  // Por defecto: mostrar checkouts de hoy y reservas vencidas (checkout pasado s
  const isToday = res.checkOutDate === today;
  const isOverdue = res.checkOutDate < today;
  return matchesSearch && (isToday || isOverdue);
});
```

Agregar estado y botón toggle:

```
const [showAllCheckouts, setShowAllCheckouts] = useState(false);

<Button
  variant={showAllCheckouts ? "default" : "outline"}
  size="sm"
  onClick={() => setShowAllCheckouts(!showAllCheckouts)}
  data-testid="button-toggle-all-checkouts"
>
  {showAllCheckouts ? "Solo hoy" : "Ver todos"}
</Button>
```

Agregar badge indicador en cada fila — verde si checkout es hoy, naranja si es pasado:

```
{res.checkOutDate === today ? (
  <Badge className="text-xs bg-green-100 text-green-800">Hoy</Badge>
) : res.checkOutDate < today ? (
  <Badge className="text-xs bg-orange-100 text-orange-800">Vencido</Badge>
) : null}
```

Aplicar `formatDateAR` en las fechas mostradas en la lista.

3c — GroupsPage: ocultar grupos finalizados/viejos por defecto

Agregar filtro para ocultar grupos con checkout anterior a hoy, a menos que se active el historial:

```
const today = new Date().toLocaleDateString("en-CA", { timeZone: "America/Argentina/Buenos_Aires" });
const [showGroupHistory, setShowGroupHistory] = useState(false);

const filteredGroups = groups?.filter((group) => {
  const matchesSearch = ... // existente sin cambios
  const matchesStatus = ... // existente sin cambios

  // Ocultar grupos ya finalizados/cancelados con fechas pasadas
  if (!showGroupHistory) {
    const isOld = group.checkOutDate < today && ["finished", "cancelled"].include(
      group.status
    );
    if (isOld) return false;
  }

  return matchesSearch && matchesStatus;
});
```

Agregar botón toggle junto a los otros filtros:

```
<Button
  variant={showGroupHistory ? "default" : "outline"}
  size="sm"
  onClick={() => setShowGroupHistory(!showGroupHistory)}
  data-testid="button-toggle-group-history"
>
  <History className="h-4 w-4 mr-1" />
  {showGroupHistory ? "Ocultar historial" : "Ver historial"}
</Button>
```

Resumen de archivos modificados

Archivo	Cambio
shared/schema.ts	Agregar <code>checkin_today</code> a <code>PlanningCellStatus</code>
server/storage.ts	Asignar <code>checkin_today</code> en ambas implementaciones de <code>getPlanningData</code>
client/src/pages/planning.tsx	Color/label <code>checkin_today</code> , <code>handleCellClick</code> abre form completo, montar <code>ReservationFormDialog</code> nuevo
client/src/pages/reservations.tsx o donde esté <code>ReservationFormDialog</code>	Agregar prop <code>defaultValues</code>
client/src/pages/check-in.tsx	<code>formatDateAR</code> en fechas
client/src/pages/check-out.tsx	Filtro hoy/vencido, toggle ver todos, <code>formatDateAR</code> , badge estado
client/src/pages/groups.tsx	Ocultar grupos viejos por defecto, toggle historial